Data Structures

BS (CS) _Fall_2025

Lab_16 Task

Learning Objectives:

1. Open Ended Lab

Project Yggdrasil: *The High-Frequency Trading Engine*

Scenario:

You have just been hired by *QuantEdge*, a high-frequency trading firm. The firm is losing millions because their current order matching engine is too slow. When a stock price changes, their system takes linear time O(n) to find the best buy/sell orders, which is unacceptable in a market that moves in microseconds. This is where you come in...

Your task is to build the core logic for **Yggdrasil**, a simplified Limit Order Book (LOB) engine. The system must accept buy and sell orders and match them instantly based on price and time priority.

Functional Requirements:

Your system must handle a stream of incoming operations. You are free to design the class structure and use any combination of Data Structures that you have learnt in the class and labs but you must implement the following functionalities efficiently:

A. Place Order

- **Input:** (orderID, type, price, quantity)
  - type: BUY or SELL
- **Logic:**
  - A **BUY** order looks for the lowest available **SELL** price.
  - A **SELL** order looks for the highest available **BUY** price.
  - If a match is found (Buy Price >= Sell Price), execute the trade (decrement quantities).
  - If the order is not fully filled, the remaining quantity must be stored in the "Book" for future matching.
- **Constraint:** You must be able to retrieve the "Best Bid" (highest buy) and "Best Ask" (lowest sell) in O(1) or O(log n) time.

B. Cancel Order

- **Input:** (orderID)
- **Logic:** Remove an order from the system immediately, regardless of its price or position in the queue.
- **Constraint:** This is the tricky part. You must be able to find and delete an arbitrary order in O(1) or O(log n). Searching through a list is not allowed.

C. Modify Order

- **Input:** (orderID, newQuantity)
- **Logic:** Update the quantity of an existing order. If the quantity increases, the order usually loses its time priority (moves to the back of the queue for that price).

D. Market Snapshot

- **Input:** K (integer)

- **Logic:** Return the top K levels of the book (e.g., the 5 highest buy prices and 5 lowest sell prices) to visualize the market depth.

Sample Data Flow:

Input:
PLACE_ORDER(ID=101, TYPE=BUY, PRICE=150, QTY=100)
PLACE_ORDER(ID=102, TYPE=BUY, PRICE=152, QTY=50)
PLACE_ORDER(ID=103, TYPE=SELL, PRICE=148, QTY=40)
CANCEL_ORDER(ID=101)
PRINT_BOOK()

Expected Logic Trace:

1. **ID 101** sits in the Buy Book at 150.
2. **ID 102** sits in the Buy Book at 152. (Best Bid is now 152).
3. **ID 103** (Sell at 148) enters. It sees the Best Bid (152). Match!
    - Trade occurs at 152. ID 102 reduces to 10 qty. ID 103 is fully filled.
4. **ID 101** is cancelled. Removed from book.
5. **Print:** Best Bid: 152 (Qty: 10). Best Ask: Empty.

Grading Criteria:

You are required to submit a Report in PDF format that has all the necessary components i.e. explanation for your solution, output screenshots, justification for the used data structures.

Key requirements include:

- Correct solution (orders match correctly (Buy >=Sell))
- Efficiency: Add, Match, and Cancel must be better than O(n)
- Architecture Justification of Data Structures chosen
- Code Quality Modular, readable, and handles edge cases (e.g., self-match)

Note: Any use of **Generative AI is strictly prohibited**, however as the lab is open ended so you can use the internet for research purposes. Any form of plagiarism will be delt strictly.